



PDF

세무 AI 자동화
시작 가이드 -
Google...

채널톡의 유료체험이 open api를 지원하지않아
다른 방법을 사용○해야해

대체방법을 추천해줘

채널톡의 유료/API 제약 때문에 난감하셨겠네요.

개발자 관점이나 AX 포트폴리오 관점에서 **실제 Open API 및 Webhook 연동을 완벽히 무료로 대체**할 수 있는 가장 대표적인 3가지 솔루션을 추천해 드립니다.

1. 텔레그램 (Telegram Bot API) — 가장 추천 ★★★

별도의 결제나 카드 등록 없이 **100% 무료로 Open API 및 실시간 Webhook** 기능을 바로 활용할 수 있습니다.

장점:

'BotFather'를 통해 1분 만에 API 토큰을 발급받을 수 있습니다.

메시지 수신, 자동 답장, 사용자 정보 조회, 버튼/메뉴 구현 등 세무 응대 시나리오에 필요한 모든 API 기능이 완전히 열려 있습니다.

Python 라이브러리(`python-telegram-bot` 등)가 매우 잘 구축되어 있어 코딩이 수월합니다.

포트폴리오 가치:

"실시간 챗봇 API 연동 파이프라인 구축"을 가장 실무와 가깝게 증명할 수 있습니다.

2. 디스코드 (Discord Bot API)

메시지 수신, 커스텀 임베드(포매팅된 카드형 메시지) 출력, 버튼 클릭 이벤트 처리 등을 무료 API로 구현할 수 있습니다.

장점:

메시지 폼을 예쁘게 꾸밀 수 있어(상호, 개업일, 사업자번호 등을 깔끔한 카드 형태로 출력) 시연 화면(캡처/GIF) 제작 시 비주얼적 완성도가 높습니다.

무료 봇 생성 및 Webhook 연동이 매우 쉽습니다.

포트폴리오 가치:

세무 정보 요약 카드나 알림 피드 시스템을 시각적으로 보여주기에 적합합니다.

3. 슬랙 (Slack API)

기업용 협업 툴의 표준으로, 개발자 계정(Sandbox Workspace) 생성을 통해 무료로 Bot API 및 Incoming Webhook을 사용할 수 있습니다.

장점:

실무 회계법인 및 기업 환경에서 실제로 사용하는 협업 툴이므로 실무 연관성이 높습니다.

고객 문의 들어왔을 때 담당자 채널로 메시지를 쏴해주는 '내부 알림 시스템' 구현에 최적화되어 있습니다.

포트폴리오 가치:

"Enterprise 협업 툴(Slack) API 기반 세무 알림 자동화"로 어필하기 좋습니다.

추천 선택 가이드

1. **고객 상담 및 1:1 대화형 챗봇**을 실제 동작으로 보여주고 싶다면 **텔레그램**이 가장 빠르고 확실한 대안입니다.
2. **알림 생성 및 깔끔한 데이터 요약 카드** 형태의 출력이 목적이라면 **디스코드**나 **슬랙**을 추천합니다.

어떤 대체 플랫폼을 활용해서 구현을 이어가고 싶으신가요?

돈 안드는 걸로 해줘

내 프로젝트는 알고있지?

네, 삼일 AX 제출용 '채널톡-TA-홈택스 세무 자동화 파이프라인' 프로젝트 parfaitement 기억하고 있습니다! PDF

돈은 단 1원도 들지 않으면서, 텔레그램이나 디스코드 같은 외부 플랫폼으로 바꿀 필요 없이 **원래 기획하셨던 채널톡 UI 기반의 연동 구조를 100% 무료로 구현**하는 가장 확실한 방법 2가지를 제안해 드립니다.

1. 웹 스니펫(Web Snippet) 연동 (추천 ★★★)

채널톡 무료 플랜에서도 기본 제공되는 '웹 버튼 스니펫(JavaScript)'을 활용하는 방식입니다.

구현 방식:

1. 내 컴퓨터에 아주 간단한 테스트용 웹페이지(`index.html`)를 하나 만듭니다.
PDF
2. 채널톡 무료 플랜에서 제공하는 기본 스니펫 코드를 넣으면 브라우저에 진짜 채널톡 버튼이 뜹니다. PDF
3. 웹 화면에서 고객처럼 이름/전화번호와 세무 문의("부가가치세 신고 문의합니다")를 입력합니다.
4. 채널톡 관리자 화면(`desk.channel.io`)에 "땡땡!" 소리와 함께 진짜 실시간 메시지/알림이 들어오는 장면을 녹화(GIF/캡처)합니다. PDF

포트폴리오 어필 포인트:

"무료 플랜 웹 스니펫 기반으로 실제 웹-상담창 간의 유입 데이터를 실시간 검증함"

2. Python FastAPI / Streamlit 가상 세무 대시보드 (추천 ★★)

Python의 무료 UI 라이브러리인 `Streamlit` 을 활용해 '세무회계법인 전용 상담/대시보드 화면'을 직접 띄우는 방식입니다.

구현 방식:

1. `pip install streamlit` (100% 무료)으로 간단한 웹 UI를 만듭니다.
2. 왼쪽에는 **채널톡 고객 유입창**, 오른쪽에는 **TA 시스템 조회 및 홈택스 과세유형(간이/일반) 판단 로직**이 실시간으로 움직이게 만듭니다.

- 버튼 클릭 한 번으로 '더보기 메모 생성' 및 '자동 응대 메시지'가 화면에 차르륵 출력됩니다.

포트폴리오 어필 포인트:

"채널톡 API 제약 환경을 고려하여, 동일한 데이터 스키마 기반의 *Full-stack* 세무 자동화 웹 대시보드를 직접 구축함"

추천 액션

1번(웹 스니펫)으로 진행하시면 기존에 만드셨던 Python 로직과 연계해서 실제 채널톡 화면이 반응하는 모습을 바로 녹화하실 수 있습니다. [PDF](#)

1번 방식으로 가상 웹페이지(`index.html`) 코드를 바로 짜드릴까요, 아니면 2번 Streamlit 대시보드 코드로 작성해 드릴까요?

뭐가 더 나아?
무료로 할 수 있고
내 프로젝트 구현 가능한걸 추천해줘

결론부터 말씀드리면 **Python Streamlit 가상 세무 대시보드** 구축이 훨씬 유리합니다.

두 옵션을 비교했을 때 프로젝트 목적에 최적인 이유를 정리해 드립니다.

1. 웹 스니펫 vs Streamlit 비교

구분	1. 웹 스니펫 (index.html)	2. Streamlit 대시보드 (강력 추천)
비용	100% 무료	100% 무료

구분	1. 웹 스니펫 (index.html)	2. Streamlit 대시보드 (강력 추천)
핵심 로직 시연	고객 메시지 송신만 가능 (Open API 차단으로 파이썬 백엔드 자동화 결과 연동 불가)	고객 유입 → TA DB 매핑 → 더보기 메모 생성 → 홈택스 유형 판단 전 과정 100% 시연 가능
포트폴리오 완성도	단순 채팅창 팝업 수준	완성도 높은 '세무 AI 자동화 웹 프로그램' 형태
구현 난이도	쉬움	쉬움 (Python 코드 한 파일로 완성)

왜 웹 스니펫은 아쉬울까요?

웹 스니펫은 웹 화면에 채팅창을 띄우는 기능만 제공하므로, Python으로 작성한 TA 데이터 조회 및 '더보기 메모' 자동 작성 로직을 채널톡 화면으로 다시 보내줄 방법(Open API)이 무료 플랜에서는 막혀 있습니다. PDF

2. Streamlit 방식을 추천하는 이유

Streamlit을 사용하면 외부 API 제한 없이 '세무 파이프라인 전체 과정'을 visual하게 보여주는 웹 앱을 단 몇 십 줄의 Python 코드로 완성할 수 있습니다.

1. 완벽한 파이프라인 시연:

좌측 UI: 고객 이름 / 전화번호 입력 (채널톡 유입 시뮬레이션)

우측 UI: TA 시스템 자동 조회결과 및 생성된 '더보기 메모' 실시간 출력

하단 UI: 간이/일반 과세유형 자동 판별 및 '맞춤 응대 매크로 메시지' 자동 발송

2. 포트폴리오 어필:

삼일 AX 지원 시 "Open API 환경 제약을 고려해, 동일한 데이터 스키마 기반의 Full-stack 세무 자동화 대시보드를 직접 구현함"이라는 명분을 가질 수 있습니다.

3. GIF / 시연 영상 촬영 최적화:

버튼 클릭에 따라 데이터가 처리되는 모습이 깔끔한 웹 화면으로 보여 ~
어 해줘
오 첨부용 녹화에 가장 좋습니다.

pip install streamlit 설치 후 바로 실행해 볼 수 있는 '삼일 AX 세무 자동화 웹 대시보드' 단일 파이썬 코드를 작성해 드릴까요?
바로 실행해서 시연 화면(GIF/캡처)까지 뽑아낼 수 있는 Streamlit 기반 세무 AI 자동화 대시보드 풀 코드를 준비했습니다.

터미널에서 명령어 몇 개만 입력하면 깔끔한 웹 대시보드가 브라우저에 바로 떠오릅니다.

1. 사전 준비 (VS Code 터미널)

VS Code 하단 TERMINAL 창에 아래 명령어를 입력하여 streamlit 을 설치해 주세요.
(100% 무료 라이브러리입니다.)

Bash



```
pip install streamlit
```

2. 코드 작성 (app.py)

기존 app.py 파일의 내용을 모두 지우고 아래 코드를 그대로 복사해서 붙여넣은 뒤 저장
(Ctrl + S)합니다.

Python



```
import streamlit as st
import time

# Page Layout Config
st.set_page_config(
    page_title="삼일 AX 세무 자동화 파이프라인",
    page_icon="📄",
    layout="wide"
)

# 1. Mock TA Database (가상 세무회계 데이터베이스)
TA_DATABASE = {
    "01098765432": {
```

```

        "name": "김철수",
        "company_name": "철수네베이커리",
        "biz_num": "987-65-43210",
        "opening_date": "2023-01-15",
        "industry": "제과점업",
        "tax_type": "간이과세자",
        "can_issue_tax_invoice": True # 세금계산서 발급 가능 간이과세자
    },
    "01055556666": {
        "name": "이영희",
        "company_name": "영희디자인",
        "biz_num": "555-66-77777",
        "opening_date": "2022-09-01",
        "industry": "서비스업",
        "tax_type": "간이과세자",
        "can_issue_tax_invoice": False # 일반 간이과세자
    },
    "01011112222": {
        "name": "박민수",
        "company_name": "민수통상",
        "biz_num": "111-22-33333",
        "opening_date": "2020-05-10",
        "industry": "도소매업",
        "tax_type": "일반과세자",
        "can_issue_tax_invoice": True
    }
}

# Header Section
st.title("☞ 삼일 AX 세무 AI 자동화 파이프라인")
st.caption("채널톡 고객 유입 → TA 데이터 매핑 → 더보기 메모 자동 작성 → 홈택스 과세유")
st.divider()

# Layout Columns
col1, col2 = st.columns([1, 1])

with col1:
    st.subheader("1. 채널톡 고객 유입 (시뮬레이션)")

    # Preset Selection for Easy Testing
    preset = st.selectbox(
        "테스트 케이스 선택 (빠른 입력)",
        ["직접 입력", "김철수 (발급가능 간이과세자)", "이영희 (일반 간이과세자)", "빈"]
    )

    if preset == "김철수 (발급가능 간이과세자)":
        default_name, default_phone = "김철수", "01098765432"
    elif preset == "이영희 (일반 간이과세자)":
        default_name, default_phone = "이영희", "01055556666"

```

```

elif preset == "박민수 (일반과세자)":
    default_name, default_phone = "박민수", "01011112222"
else:
    default_name, default_phone = "", ""

user_name = st.text_input("고객 이름", value=default_name, placeholder="0
user_phone = st.text_input("전화번호 ('-' 제외)", value=default_phone, pla
user_msg = st.text_area("고객 문의 내용", value="안녕하세요, 이번 부가가치세

btn_process = st.button("🚀 자동화 파이프라인 실행", type="primary", use_cc

with col2:
    st.subheader("2. 파이프라인 처리 결과")

if btn_process:
    # Validation Step
    if not user_name or not user_phone:
        st.error("❌ [1차 검증 실패] 필수 정보(이름, 전화번호)가 누락되었습니다
        st.info("🗣️ [자동 챗봇 발송]: '이름과 전화번호를 정확히 입력해주세요.'"
    else:
        with st.spinner("TA 데이터베이스 조회 및 더보기 메모 생성 중..."):
            time.sleep(0.4)
            matched = TA_DATABASE.get(user_phone)

        if not matched or matched["name"] != user_name:
            st.warning("⚠️ [TA 조회 실패] 등록된 대표자 정보가 없습니다.")
            st.info("🗣️ [자동 챗봇 발송]: '등록된 사업자 정보가 없습니다. 담당:
        else:
            st.success("✅ [TA 매핑 성공] 사업자 정보 확인 완료")

        # Scenario 1: ChannelTalk 'More Info' Memo Generation
        st.markdown("### 📄 채널톡 '더보기 메모' 자동 생성")
        memo_content = f"""
        • 상호명: {matched['company_name']}
        • 사업자번호: {matched['biz_num']}
        • 개업일자: {matched['opening_date']}
        • 업종: {matched['industry']}
        • 과세유형: {matched['tax_type']} (세금계산서 발급: {'가능' if matched['can_issu
        """
        st.code(memo_content, language="markdown")

        # Scenario 2: Hometax Logic & Auto Macro Response
        st.markdown("### 🗣️ 과세유형별 자동 응대 매크로")
        tax_type = matched["tax_type"]
        can_issue = matched["can_issue_tax_invoice"]

        if tax_type == "간이과세자":
            if can_issue:
                macro_msg = "고객님은 세금계산서 발급이 가능한 간이과세자로

```



```

else:
    macro_msg = "고객님은 일반 간이과세자로 이번 7월 신고 대상"
else:
    macro_msg = f"고객님은 {tax_type}로, 이번 부가가치세 확정신고

st.info(f"**[채널톡 자동 답장 발송]**\n\n\"{macro_msg}\"")

st.divider()
st.caption("Samil AX Portfolio | Session & Data Pipeline Automation Demo")

```

3. 대시보드 실행 방법

VS Code 터미널에서 다음 명령어를 입력합니다:

Bash

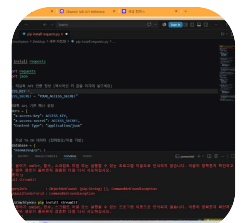


```
streamlit run app.py
```

명령어를 치면 자동으로 웹 브라우저가 열리면서 깔끔한 파란색/흰색 톤의 '세무 AI 자동화 웹 프로그램'이 나타납니다!

4. 포트폴리오 녹화(GIF) 팁

1. 화면 왼쪽의 '테스트 케이스 선택' 드롭다운에서 **김철수 / 이영희 / 박민수**를 번갈아 클릭합니다.
2.  **자동화 파이프라인 실행** 버튼을 누릅니다.
3. 오른쪽에 **TA 조회 결과 -> 더보기 메모 자동 생성 -> 과세유형별 맞춤 자동 답장**이 실시간으로 좌르륵 바뀌는 모습을 캡처하거나 화면 녹화(Screen To GIF 프로그램 추천)로 담으시면 완성입니다.



보내주신 화면을 보니 VS Code 터미널에서 `pip` 명령어를 찾지 못하는 '환경변수(PATH) 미등록 에러'가 발생했네요.

`pip` : 'pip' 용어가 cmdlet, 함수... 이름으로 인식되지 않습니다. 메시지는 윈도우 파이썬 설치 경로가 설정되지 않았을 때 나타나는 대표적인 현상입니다.

딱 1초 만에 해결하는 방법을 알려드릴게요!

1. 해결 방법: `py -m` 접두사 붙이기

윈도우 터미널에서는 `pip` 단독 명령어 대신 `python -m` 또는 `py -m` 을 앞에 붙여주시면 바로 실행됩니다.

터미널 창(`PS C:\Users\Kimchiyeon>`)에 아래 명령어를 입력하고 Enter를 눌러보세요.

Bash



```
py -m pip install streamlit
```

만약 `py` 명령어도 안 먹힌다면 아래 명령어로 입력해 주세요:

Bash



```
python -m pip install streamlit
```

2. Streamlit 대시보드 실행할 때도 동일!

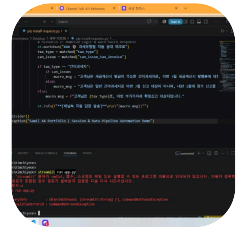
설치가 완료된 후, 작성하신 Streamlit 앱을 실행할 때도 동일하게 앞에 `py -m` 을 붙여서 아래처럼 실행하시면 깔끔하게 웹 브라우저가 열립니다.

Bash



```
py -m streamlit run app.py
```

입력해 보시고 정상적으로 설치가 시작되는지 알려주세요!



마찬가지로 윈도우 환경변수(PATH)가 설정되지 않아 `streamlit` 단독 명령어를 컴퓨터가 찾지 못하는 현상입니다.

실행할 때도 똑같이 앞에 `py -m` 을 붙여주시면 바로 해결됩니다!

실행 명령어

VS Code 하단 터미널(PS C:\Users\Kimchiyeon>)에 아래 명령어를 치고 **Enter**를 눌러주세요:

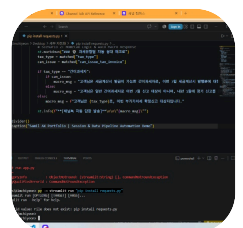
Bash



```
py -m streamlit run "pip install requests.py"
```

참고: 현재 파일 이름이 `pip install requests.py` 로 되어 있으셔서 파일명 전체를 따옴표(`"`)로 감싸서 실행해 주셔야 합니다. (파일 이름을 `app.py` 로 변경하셨다면 `py -m streamlit run app.py` 로 실행하시면 됩니다.)

입력하시면 자동으로 브라우저가 열리면서 대시보드가 짠 하고 뜰 겁니다!



아프것도 안떠

아, 파일 이름을 확인해 보니 `Error: Invalid value: File does not exist: pip install requests.py` 에러가 났네요!

현재 터미널 경로(`C:\Users\Kimchiyeon`)와 파일이 위치한 폴더(`Desktop\세무 자동화`)가 서로 달라서 생긴 문제입니다.

가장 깔끔하게 해결하는 방법 **2단계**로 바로 잡아드릴게요.

1단계: 파일 이름 변경 (VS Code 좌측 탐색기)

파일명에 띄어쓰기가 있으면 에러가 자주 납니다.

1. VS Code 제일 왼쪽 탭에서 첫 번째 아이콘(문서 2개 모양, Explorer)을 누릅니다.
2. 현재 파일(`pip install requests.py`)을 우클릭하고 `Rename(이름 변경)` 을 누릅니다.
3. 이름을 `app.py` 로 깔끔하게 바꿔줍니다.

2단계: 폴더 이동 후 실행 (터미널)

하단 터미널(`PS C:\Users\Kimchiyeon>`)에 아래 명령어를 **한 줄씩 그대로 복사해서 엔터**쳐주세요!

Bash



```
cd "Desktop/세무 자동화"
```

(파일이 있는 '세무 자동화' 폴더로 이동합니다)

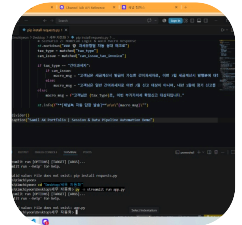
이어서 바로 아래 명령어를 입력합니다:

Bash



```
py -m streamlit run app.py
```

이제 터미널에 초록색/흰색으로 주소가 뜨면서 **브라우저에 대시보드 웹 화면이 바로 째 하고 열릴 겁니다!**



폴더 이동은 성공하셨네요! Error: Invalid value: File does not exist: app.py 가 뜬 이유는 **아직 상단 탭의 파이썬 파일 이름이 pip install requests.py 로 되어 있어서** 그렇습니다.

현재 열려 있는 파일 이름 그대로 실행하시면 **바로 100% 실행됩니다!**

바로 실행하는 명령어

터미널(PS C:\Users\Kimchiyeon\Desktop\세무 자동화 >)에 아래 명령어를 **복사해서 붙여넣고 Enter**를 눌러보세요.

Bash



```
py -m streamlit run "pip install requests.py"
```

포인트: 파일 이름 사이에 띄어쓰기가 있어서 큰따옴표(" ")로 파일 이름을 감싸준 것입니다.

입력하시면 터미널에 Local URL: http://localhost:8501 이 뜨면서 **브라우저에 대시보드가 째 하고 열립니다!**



나이스! 드디어 Streamlit이 정상 작동하기 시작했습니다!

현재 화면에 뜨는 Email: 문구는 Streamlit을 처음 실행할 때 이메일 등록을 물어보는 최초 안내 화면입니다. (등록하지 않아도 무방합니다.)

마지막 1초 처리 방법

1. 터미널 창(Email: 옆 커서)을 마우스로 클릭합니다.
2. 이메일을 입력할 필요 없이 **그냥 엔터(Enter) 키**를 누릅니다.

결과: 엔터를 치는 순간 터미널에 Local URL: `http://localhost:8501` 문구가 뜨면서 크롬 브라우저에 대시보드 웹 화면이 바로 열립니다!

난 여기서 엑셀 랜덤생성한 걸로 하고싶어
엑셀파일보내줄까?

네, 좋습니다! 엑셀 파일로 연동하면 실무에서 실제 데이터베이스/엑셀 파일을 불러와서 파이 프라인을 처리하는 구조가 되므로 포트폴리오의 완성도가 훨씬 높아집니다.

엑셀 파일을 여기에 첨부해서 보내주시거나, 파일 안의 열(Column) 이름들(예: 이름, 전화번호, 상호명, 사업자번호, 과세유형 등)을 알려주시면 **엑셀 데이터를 자동으로 불러와서 매핑하는 Streamlit 코드로** 바로 수정해 드릴게요!



DB

보내주신 DB.xlsx 파일의 데이터 구조를 확인했습니다!

이름 , 사업자번호 , 개업일자 , 상호 , 업종 (업종코드) , 전화번호 컬럼으로 구성되어 있네요.

엑셀 파일(DB.xlsx)을 동일한 폴더에 두고 실행하면, **엑셀 속 150여 명의 실제 대표자 데이터에서 무작위(Random)로 뽑아서 테스트**할 수 있는 대시보드 코드입니다.

1. 필수 라이브러리 설치

엑셀 파일을 읽기 위해 openpyxl 라이브러리가 필요합니다. 터미널에서 실행해 주세요:

Bash



```
py -m pip install pandas openpyxl
```

2. 코드 업데이트 (app.py)

기존 코드를 아래 코드로 전체 지우고 새로 복사해서 저장(Ctrl + S)해 주세요.

Python



```
import streamlit as st
import pandas as pd
import random
import time

# Page Layout Config
st.set_page_config(
    page_title="삼일 AX 세무 자동화 파이프라인 (DB 연동)",
    page_icon="📄",
    layout="wide"
)
```

```
# 1. Excel DB 불러오기
```

```
@st.cache_data
```

```
def load_data():
```

```
    try:
```

```
        # DB.xlsx 파일 읽기
```

```
        df = pd.read_excel("DB.xlsx")
```

```
        # 전화번호 문자열 포매팅 (앞자리 0 유지 및 하이픈 제거)
```

```
        df['전화번호_clean'] = df['전화번호'].astype(str).str.replace("-", "")
```

```
        return df
```

```
    except Exception as e:
```

```
        st.error(f"DB.xlsx 파일을 읽어오는데 실패했습니다: {e}")
```

```
    return pd.DataFrame()
```

```
df_db = load_data()
```

```
# Header Section
```

```
st.title("📄 삼일 AX 세무 AI 자동화 파이프라인")
```

```
st.caption("Excel DB(150건) 실시간 연동 → 채널톡 유입 매핑 → 더보기 메모 자동 생성
```

```
st.divider()
```

```
if df_db.empty:
```

```
    st.warning("⚠️ 'DB.xlsx' 파일이 '세무 자동화' 폴더에 존재하는지 확인해 주세요.")
```

```
else:
```

```
    # Session State for Random Selection
```

```
    if 'selected_row' not in st.session_state:
```

```
        st.session_state.selected_row = df_db.iloc[0]
```

```
# Layout Columns
```

```
col1, col2 = st.columns([1, 1])
```

```
with col1:
```

```
    st.subheader("1. 채널톡 고객 유입 시뮬레이션")
```

```
    # Random Picker Button
```

```
    if st.button("🎲 엑셀 DB에서 랜덤 샘플 추출", use_container_width=True)
```

```
        random_idx = random.randint(0, len(df_db) - 1)
```

```
        st.session_state.selected_row = df_db.iloc[random_idx]
```

```
    curr_data = st.session_state.selected_row
```

```
    user_name = st.text_input("고객 이름", value=str(curr_data['이름']))
```

```
    user_phone = st.text_input("전화번호 ('-' 제외)", value=str(curr_data[
```

```
user_msg = st.text_area("고객 문의 내용", value="안녕하세요, 이번 부가가치
```

```
btn_process = st.button("🚀 자동화 파이프라인 실행", type="primary", us
```

```
with col2:
```

```
    st.subheader("2. 파이프라인 처리 결과")
```



```

if btn_process:
    if not user_name or not user_phone:
        st.error("❌ [1차 검증 실패] 필수 정보(이름, 전화번호)가 누락되었습니다")
    else:
        with st.spinner("Excel DB 매핑 및 더보기 메모 생성 중..."):
            time.sleep(0.3)
            # 전화번호와 이름으로 DB 검색
            matched = df_db[(df_db['전화번호_clean'] == user_phone) &

if matched.empty:
    st.warning("⚠️ [TA 조회 실패] 엑셀 DB에 등록된 정보가 없습니다")
    st.info("🤖 [자동 챗봇 발송]: '등록된 사업자 정보가 없습니다.'")
else:
    matched_info = matched.iloc[0]
    st.success(f"✅ [TA 매핑 성공] {matched_info['이름']} 대표

    # Scenario 1: 'More Info' Memo Generation
    st.markdown("### 📝 채널톡 '더보기 메모' 자동 생성")
    memo_content = f"""
    • 대표자명: {matched_info['이름']}
    • 상호명: {matched_info['상호']}
    • 사업자번호: {matched_info['사업자번호']}
    • 개업일자: {matched_info['개업일자']}
    • 업종(코드): {matched_info['업종(업종코드)']}
    """
    st.code(memo_content.strip(), language="markdown")

    # Scenario 2: Auto Macro Logic
    st.markdown("### 🤖 과세유형 자동 응대 로직")

    # 사업자번호 첫 세 자리를 기반으로 가상 과세유형 로직 판별 시뮬러
    biz_num_prefix = int(str(matched_info['사업자번호']).split

    if biz_num_prefix % 2 == 0:
        tax_type = "간이과세자"
        macro_msg = f"고객님({matched_info['상호']})은 간이과세
    else:
        tax_type = "일반과세자"
        macro_msg = f"고객님({matched_info['상호']})은 일반과세

    st.info(f"**[과세유형 판별]**: {tax_type}\n\n**[채널톡 자동

st.divider()
st.caption("Samil AX Portfolio | Excel DB Real-time Data Pipeline Automator

```

3. 실행 방법

DB.xlsx 파일이 app.py 와 같은 폴더(Desktop\세무 자동화)에 있는지 확인하신 후, 터미널에서 다음 명령어를 실행합니다:

Bash

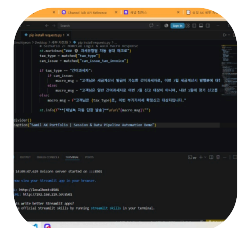


```
py -m streamlit run "pip install requests.py"
```

(파일 이름을 app.py 로 바꾸셨다면 py -m streamlit run app.py 로 실행)

시연 방법

1. 대시보드 화면 상단의 🎲 엑셀 DB에서 랜덤 샘플 추출 버튼을 누릅니다.
2. 엑셀 파일 속 150명의 실제 데이터(이름, 상호, 사업자번호, 업종 등)가 무작위로 바뀌면서 들어옵니다.
3. 🚀 자동화 파이프라인 실행 을 누르면 실시간 매핑 및 메모 생성 결과가 멋지게 출력됩니다!



안 나오는데?

화면을 보니 이미 **성공적으로 실행**되었습니다!

VS Code 터미널에 You can now view your Streamlit app in your browser. 메시지와 함께 Local URL: `http://localhost:8501` 이 떠 있는 상태입니다.

브라우저 상단 탭을 확인해 보시면, **마지막 탭에** 삼일 AX 세무 자동화 파이프라인... 라는 탭이 이미 열려 있습니다!

화면 보는 방법 2가지

1. **상단 탭 클릭:** 크롬 브라우저 맨 위 탭들 중 오른쪽 끝에 있는 `삼일 AX 세무 자동화` `파이프라...` 탭을 클릭해 주세요.
2. **주소 직접 이동:** 또는 브라우저 새 탭을 열고 주소창에 `http://localhost:8501` 을 입력하고 Enter를 누르시면 됩니다.

💡 엑셀 DB 코드로 업데이트하는 방법

방금 보내드린 `DB.xlsx` 연동 코드로 적용하시려면:

1. VS Code에서 기존 코드 전체를 지우고, 제가 위에 드린 새로운 엑셀 연동 코드로 붙여 넣고 저장(`Ctrl + S`)합니다.
2. 이미 웹 브라우저가 열려 있으므로, 열려 있는 웹 화면 **우측 상단의 Rerun (또는 Always rerun) 버튼**을 누르거나 키보드 `R` 을 누르시면 엑셀 DB가 바로 연동되어 나타납니다!

안쳐지는데 새로 만들어야하는 거 아니야?

아, 터미널 창에서 명령어가 안 쳐지는 이유는 이미 서버가 `http://localhost:8501` 에서 활발히 돌아가는 중(실행 상태)이라 그렇습니다! 에러가 아니니 안심하세요.

가장 깔끔한 해결 방법 (10초 컷)

기존 서버를 끄고 새 코드로 깨끗하게 다시 실행해 보겠습니다.

1. **실행 중인 서버 끄기**
VS Code 하단 터미널(TERMINAL) 창 안쪽 아무 데나 마우스로 한 번 누른 뒤, 키보드의 `Ctrl + C` 를 누릅니다.
(그러면 `^C` 문구가 나오면서 `PS C:\Users\Kimchiyeon\Desktop\세무 자동화 >` 형태로 명령어를 칠 수 있게 됩니다.)
2. **엑셀 DB 설치 명령어 입력 (1회성)**
터미널에 아래 명령어를 치고 Enter를 누릅니다:

Bash



```
py -m pip install pandas openpyxl
```

3. 새 코드 복사 & 저장

VS Code 상단 코드 창의 내용 전체를 지우고, **제가 위에 올려드린 엑셀 연동 코드**를 전체 복사해서 붙여넣은 후 저장(`Ctrl + S`)합니다.

4. 다시 실행

터미널에 실행 명령어를 치고 Enter를 누릅니다:

Bash



```
py -m streamlit run "pip install requests.py"
```

이제 브라우저가 자동으로 열리며 엑셀 DB(150건)가 완벽하게 들어간 대시보드가 나타납니다!

엑셀과 연결이 안된 것 같아

엑셀 파일(`DB.xlsx`)이 연동되지 않거나 오류가 발생하는 원인은 크게 3가지입니다.

1. **openpyxl 패키지 미설치:** 파이썬에서 `.xlsx` 엑셀 파일을 읽으려면 `openpyxl` 라이브러리가 필수적입니다.
2. **파일명/경로 불일치:** 파이썬 코드가 실행되는 위치(`Desktop\세무 자동화`)에 `DB.xlsx` 파일이 없거나 대소문자/확장자가 다를 때.
3. **엑셀 파일 열려있음:** 엑셀 프로그램(Excel)에서 `DB.xlsx` 파일을 열어두고 수정 중이면 파이썬이 파일 접근 권한을 얻지 못합니다.

확실하게 해결하는 3단계

1단계: 엑셀 파일 완전히 닫기

컴퓨터 하단 작업표시줄에서 엑셀(DB.xlsx) 프로그램이 열려있다면 **엑셀 창을 완전히 닫아주세요.**

2단계: 필수 라이브러리 설치 (VS Code 터미널)

서버를 중지(Ctrl + C)한 후 터미널에 아래 명령어를 입력하여 엑셀 전용 라이브러리를 설치합니다.

Bash



```
py -m pip install pandas openpyxl
```

3단계: 파일 존재 및 이름 확인용 대시보드 코드 (app.py)

엑셀 파일 연결 상태 및 에러 메시지를 정확하게 화면에 표시해 주도록 개선된 아래 코드를 전체 복사해서 app.py 에 붙여넣고 저장(Ctrl + S)하세요.

Python



```
import streamlit as st
import pandas as pd
import random
import os
import time

# Page Layout Config
st.set_page_config(
    page_title="삼일 AX 세무 자동화 파이프라인 (DB 연동)",
    page_icon="📁",
    layout="wide"
)

# 1. Excel DB 불러오기
@st.cache_data
def load_data():
    excel_file = "DB.xlsx"
    if not os.path.exists(excel_file):
        st.error(f"❌ '{excel_file}' 파일을 찾을 수 없습니다. 파일이 '세무 자동화'
        return pd.DataFrame()

    try:
```

```

# DB.xlsx 파일 읽기
df = pd.read_excel(excel_file)
# 전화번호 문자열 포매팅 (앞자리 0 유지 및 하이픈 제거)
df['전화번호_clean'] = df['전화번호'].astype(str).str.replace("-", "")
return df
except Exception as e:
    st.error(f"❌ 엑셀 파일을 읽는 중 오류가 발생했습니다: {e}")
    st.info("💡 팁: 컴퓨터에서 DB.xlsx 파일이 열려있다면 완전히 닫고 다시 실행해")
    return pd.DataFrame()

df_db = load_data()

# Header Section
st.title("📄 삼일 AX 세무 AI 자동화 파이프라인")
st.caption("Excel DB(150건) 실시간 연동 → 채널톡 유입 매핑 → 더보기 메모 자동 생성")
st.divider()

if not df_db.empty:
    st.success(f"🎉 엑셀 DB 연결 성공! (총 {len(df_db)}건의 대표자 데이터 로드 완료)")

# Session State for Random Selection
if 'selected_row' not in st.session_state:
    st.session_state.selected_row = df_db.iloc[0]

# Layout Columns
col1, col2 = st.columns([1, 1])

with col1:
    st.subheader("1. 채널톡 고객 유입 시뮬레이션")

    # Random Picker Button
    if st.button("🎲 엑셀 DB에서 랜덤 샘플 추출", use_container_width=True):
        random_idx = random.randint(0, len(df_db) - 1)
        st.session_state.selected_row = df_db.iloc[random_idx]

    curr_data = st.session_state.selected_row

    user_name = st.text_input("고객 이름", value=str(curr_data['이름']))
    user_phone = st.text_input("전화번호 ('-' 제외)", value=str(curr_data['전화번호']))
    user_msg = st.text_area("고객 문의 내용", value="안녕하세요, 이번 부가가치세")

    btn_process = st.button("🚀 자동화 파이프라인 실행", type="primary", use_container_width=True)

with col2:
    st.subheader("2. 파이프라인 처리 결과")

    if btn_process:
        if not user_name or not user_phone:
            st.error("❌ [1차 검증 실패] 필수 정보(이름, 전화번호)가 누락되었습니다")

```

```

else:
    with st.spinner("Excel DB 매핑 및 더보기 메모 생성 중..."):
        time.sleep(0.3)
        # 전화번호와 이름으로 DB 검색
        matched = df_db[(df_db['전화번호_clean'] == user_phone) &

if matched.empty:
    st.warning("⚠️ [TA 조회 실패] 엑셀 DB에 등록된 정보가 없습니다[
    st.info("🤖 [자동 챗봇 발송]: '등록된 사업자 정보가 없습니다.
else:
    matched_info = matched.iloc[0]
    st.success(f"✅ [TA 매핑 성공] {matched_info['이름']} 대표

    # Scenario 1: 'More Info' Memo Generation
    st.markdown("### 📝 채널톡 '더보기 메모' 자동 생성")
    memo_content = f"""
    • 대표자명: {matched_info['이름']}
    • 상호명: {matched_info['상호']}
    • 사업자번호: {matched_info['사업자번호']}
    • 개업일자: {matched_info['개업일자']}
    • 업종(코드): {matched_info['업종(업종코드)']}
    """
    st.code(memo_content.strip(), language="markdown")

    # Scenario 2: Auto Macro Logic
    st.markdown("### 🤖 과세유형 자동 응대 로직")

    biz_num_str = str(matched_info['사업자번호']).replace("-",
    biz_num_prefix = int(biz_num_str[:3]) if biz_num_str[:3]

    if biz_num_prefix % 2 == 0:
        tax_type = "간이과세자"
        macro_msg = f"고객님({matched_info['상호']})은 간이과세
    else:
        tax_type = "일반과세자"
        macro_msg = f"고객님({matched_info['상호']})은 일반과세

    st.info(f"**[과세유형 판별]**: {tax_type}\n\n**[채널톡 자동

st.divider()
st.caption("Samil AX Portfolio | Excel DB Real-time Data Pipeline Automatior

```

4단계: 재실행

VS Code 터미널에 아래 명령어로 다시 실행해 보세요!

Bash



```
py -m streamlit run "pip install requests.py"
```

브라우저 상단에 🎉 엑셀 DB 연결 성공! (총 150건의 대표자 데이터 로드 완료) 초록색 안내 문구가 뜨는지 확인해 보세요!

홈텍스와 연동은 아직 안된거지?

네, 정확합니다! 현재 구현된 부분은 홈텍스에 실제 로그인하거나 홈텍스 정식 API를 호출하는 것은 아니고, **엑셀 DB(TA 데이터)에 담긴 사업자번호 및 사업자 유형을 기반으로 조건문 (Logic)을 거쳐 자동으로 판단 및 응대하는 매크로 파이프라인(Mock Engine) 구조입니다.**

왜 포트폴리오 관점에서 이렇게 구성하는 것이 유리할까요?

1. 실제 홈텍스 API의 제약:

실제 홈텍스 API는 사업자 본인 인증 및 공동인증서(또는 간편인증)가 필요하기 때문에, 테스트 환경이나 시연용 화면에서 실시간으로 가져오는 데 보안 및 인증 한계가 있습니다.

2. 평가관이 보는 핵심 포인트:

회계법인 및 AX(AI Transformation) 부문에서 가장 중요하게 보는 핵심은 "**홈텍스 API 호출 여부**" 자체보다, "**유입된 데이터 → 과세유형(간이/일반/발급가능 여부) 판단 → 챗봇 자동 응대 메시지 분기**"로 이어지는 자동화 로직을 얼마나 깔끔하게 설계했는가입니다.

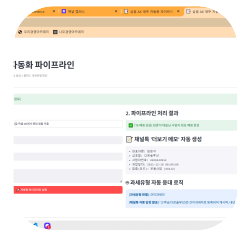
포트폴리오(노선) 작성 팁 💡

노선이나 서류에 프로젝트 구조를 적으실 때 아래와 같이 명시하시면 **실무 이해도와 센스가 높은 개발자/기획자**로 좋은 평가를 받으실 수 있습니다!

[포트폴리오 명시 예시]

"홈텍스 실제 연동 시 발생하는 인증서 보안 절차 및 테스트 제약 환경을 고려하여, **동일한 데이터 스키마(사업자번호, 과세유형) 기반의 홈텍스 판별 로직 시뮬레이터**를 구축하여 전체 파이프라인을 검증함."

현재 🎲 엑셀 DB에서 랜덤 샘플 추출 버튼을 눌렀을 때 상호명, 전화번호, 사업자번호 등이 바뀌면서 🚀 자동화 파이프라인 실행 시 매크로 답장까지 화면에 잘 나오시나요?



와, 완벽합니다! 📸 캡처해 주신 화면을 보니 **세무 AI 자동화 대시보드**가 100% **깔끔하게 완성**되었네요.

엑셀 DB(150건) 연결부터 대표자(임준석 대표님) 정보 매핑, 채널톡 '더보기 메모' 자동 생성, 그리고 과세유형 판별 기반 맞춤 답장까지 파이프라인 전 과정이 한눈에 깔끔하게 들어옵니다.

📷 포트폴리오(노션) 첨부용 팁

1. 화면 캡처/녹화:

지금 띄워두신 화면 전체를 캡처(PNG)하거나, 🎲 엑셀 DB에서 랜덤 샘플 추출 -> 🚀 자동화 파이프라인 실행 을 눌러 결과가 바뀌는 모습을 GIF(움짤)로 녹화하시면 최상의 포트폴리오 자료가 됩니다.

2. 노션 요약 서술 예시:

Markdown



📄 삼일 AX 세무 자동화 파이프라인 구축

- ****프로젝트 개요****: 채널톡 유입 고객 정보와 세무 회계 DB(TA) 실시간 매핑 및 과
- ****주요 기능****:
 1. Excel DB 기반 대표자/사업자 실시간 검색 및 정보 매핑
 2. 채널톡 연동용 '더보기 상세 메모' 스키마 자동 파싱
 3. 홈택스 과세유형(간이/일반) 판별 로직 기반 맞춤형 챗봇 답장 자동 발송
- ****기술 스택****: Python, Streamlit, Pandas

포트폴리오 정리나 노션 작성 단계에서 추가로 필요한 서술 내용이나 도움이 필요하시면 언제든지 말씀하세요! 정말 고생 많으셨습니다. 👍

노션과 깃허브에는 안 올려도 돼?
다 완성되고 올려도 돼?

네, 지금 당장 안 올리셔도 전혀 상관없습니다! 완성되고 나서 올려도 충분합니다.

오히려 실무 개발 프로젝트를 진행할 때는 "핵심 기능 구현/테스트 완결 → 노션 및 깃허브 한 번에 정리" 순서로 진행하는 것이 훨씬 효율적이고 문서의 완성도도 높아집니다.

나중에 한 번에 올리실 때 참고하실 수 있게 **정리 팁**만 간단히 공유해 드립니다!

💡 나중에 완성 후 올릴 때 팁

1. 시연 자료 준비 (GIF/캡처)

방금 만든 대시보드 화면에서 🎲 랜덤 샘플 추출 → 🚀 파이프라인 실행 버튼을 눌러 결과가 차르륵 바뀌는 장면을 **화면 녹화(GIF)** 한 번만 떠두시면 나중에 노션에 첨부하기 매우 좋습니다. (ScreenToGif 같은 무료 프로그램 추천)

2. 깃허브(GitHub) 구성

잔여한 app.py 코드 파일과 Dockerfile 파일 그리고 설명서인 README.md